

TNPG: Trois

Roster: Ricky Lin, Ethan Saldanha, Hannah Grimskog-Tran, Tudor Ganshaw

TARGET SHIP DATE: 2026-04-22

Project Description:

- Users are given a game description/information and have to guess the total sales, then are shown the actual ranking/sales. The graph grows as more rounds continue.
- Users can rank/review their favorite games, which will be shown on their own profile page and a page for the game. They can also favorite games and add bios to show other users what they are like.
 - Their reviews may be added to the existing data and will update statistics accordingly
- Users can see top rated games with graphs to show accurate data

Program Components:

Frontend:

- HTML Templates:
 - Login
 - Logout
 - Register
 - Profile
 - Will show users' reviews, favorite games, and a chosen bio
 - Home
 - Guessing Game
 - Page where the users will be given game info & try to guess sales numbers. Top of the page will have a graph showing guess vs reality, will update as more games are guessed
 - Global Ranking
 - Shows top rated games
 - Game Info
 - Shows info and user reviews for the game
- CSS:
 - Tailwind

Backend Components:

- Flask app (__init__.py) through Jinja2
- Database through SQLite3
- API
 - RAWG API

Front-End Framework: Tailwind

We decided to use Tailwind as our front end framework because it's easiest to customize visually, as opposed to Bootstrap and the other one that focus more on adding features like dropdown menus, image carousels, etc.

Dataviz Library: Chart JS

Easier than D3 and for our purposes, perfectly sufficient, thus giving us more time to debug and otherwise improve on our code.

APIs:

- Video games API:
 - <https://rawg.io/apidocs> [In KB]
 - Provides information about games like game descriptions and ratings

Datasets:

- Video game sales
 - <https://zenodo.org/records/5898311>

Roles:

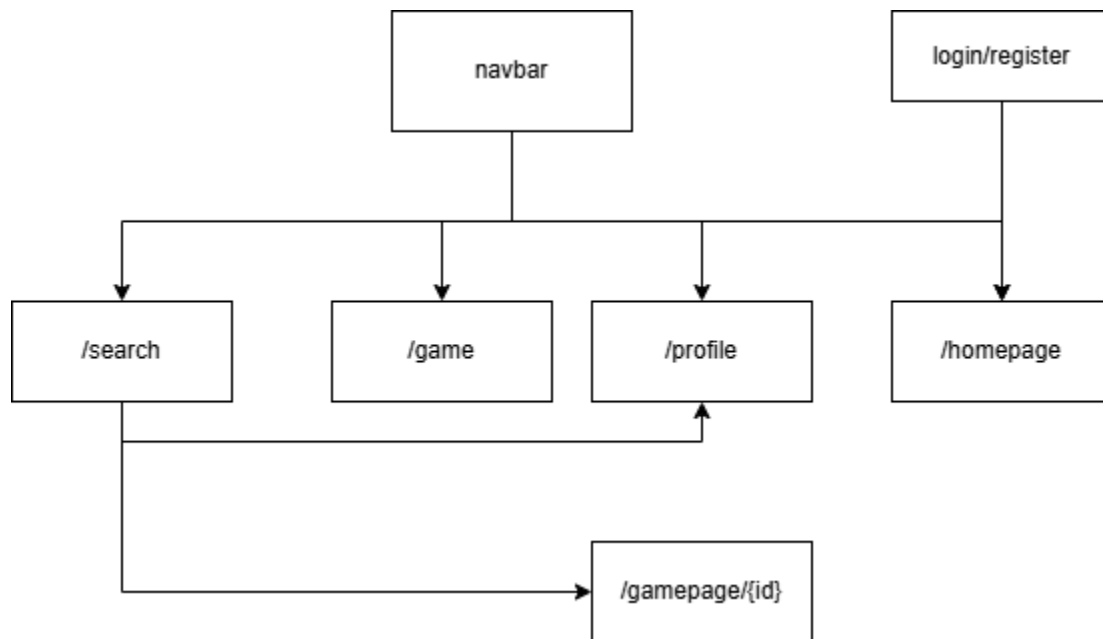
Ricky (PM): Database functions, frontend

Ethan: User handling (creating profiles, making reviews, etc)

Hannah: Frontend, [graphs](#)

Tudor: Database sanitation, APIs, backend

Sitemap:



Database Organization:

We are storing our data in a .json file in our repo. Our populate_db.py will send information from the .json file to the database that is accessible through various python files by using SQLite3. The data is then passed to the frontend through flask, where then, javascript and chart.js will be able to see it.

DB/JSON Details:

(games.json is static, populated once and unchanging. games.db is used to update user rating & reviews)

Games.json
Rank
Name
Consoles
Year
Publisher

Games.json
NA Sales
EU Sales
JP Sales
Other Sales
Global Sales
Description
Public Rating (Metacritic)
ID (rank)

Games.db	
name	TEXT
reviews	TEXT
user_rating	INT
num_ratings	INT
id	INT (PK)

Users.db	
username	TEXT
password	TEXT
reviews	TEXT
bio	TEXT
favorites	TEXT

Users.db	
id	INT (PK)

Reviews.db	
game_id	INT
body	TEXT
user_id	INT
id	INT (PK)